

≡

Medium

🔍 Search

✍ Write

🔔

👤

🏠 Home

📖 Library

👤 Profile

📄 Stories

📊 Stats

👤 Following

💡 Dev Genius

🐍 Python in Plain En...

🧠 Generative AI

🖥 Realworld AI Use C...

📺 The Useful Tech

🎧 Level Up Coding

🧠 AI Mind

🌀 Deep concept

📚 Top Python Librari...

🧙 The Mac Alchemist

⌵ More

If AI Writes Your Code, Why Use Python?

Noah Mitchem

Follow

7 min read · Apr 28, 2026

👏 1.2K

💬 47

↺ 3

🔖

🎥

📄

⋮



The modern python stack.

☐ Reading list

☐ Media Generation

☐ Implement

☐ Python

☐ Business Opportunities

Create new list

For the last decade, fast-to-ship beat fast-to-run. Not anymore.

Picking a language for a new project was usually an easy answer. You used Python or TypeScript because the ecosystems were enormous, the hiring pool was deep, and you could make an impressive demo by Friday. Rust, Go, C++, and many more would give you 10–100x the performance, but you paid for it: six months of ramp, a smaller talent market, a build system that fought you. So you shipped the Python version, sold it to customers, and promised yourself you’d “make it performant later.” You rarely did, and that was fine, because nobody else did either.

That bargain is over, and it’s over because AI got good at the hard languages.

https://medium.com/@NMitchem/if-ai-writes-your-code-why-use-python-bf8c4ba1a055

Page 1 of 7

The hard languages got easy first

Two years ago, GPT-4 couldn't write a Rust function without hallucinating crate names. By April of 2026, Claude Opus 4.7, GPT-5.5, Gemini 3.1, and DeepSeek V4 had all cleared 80% on SWE-bench Verified within weeks of each other. The labs are openly optimizing for systems work: concurrency bugs, race conditions, and architectural flaws identified during the planning phase.

The best one-tweet explanation came from [CtrlAltDwayne](#) last month:

"The best argument for Rust in 2026 is not memory safety or performance. It is that AI writes better Rust than it writes C++. The compiler feedback loop is so tight that models self-correct in real time. Every error message is a free training signal. Rust was accidentally designed for AI-assisted development 10 years before anyone knew that mattered." via [X.com](#)

Q 1

The same logic applies, to varying degrees, to Go and Swift. Strong type systems plus fast compile-and-check loops give agents the tightest iteration cycle. The systems languages that were hardest for humans turn out to be the easiest for agents.

What actually shipped

Look at what landed in a single quarter.

Microsoft rewrote the TypeScript compiler in Go. The team behind the most-used superset of JavaScript shipped [TypeScript 7.0 beta](#) last week, roughly 10x faster than 6.0, by porting the decade-old TypeScript codebase to Go. [Anders Hejlsberg's reasoning](#): Go delivered most of the performance benefit at a fraction of the engineering cost. The biggest JS/TS shop on earth picked a harder, faster language for its flagship tool, and they did it because the effort calculus changed under them.

Nicholas Carlini, a researcher at Anthropic, [orchestrated 16 parallel Claude agents](#) to write a production C compiler in Rust. 100,000 lines. It boots Linux 6.9 on x86, ARM, and RISC-V. It compiles QEMU, FFmpeg, SQLite, PostgreSQL, and Redis. It runs Doom. Total cost: just under \$20,000 across nearly 2,000 Claude Code sessions.

Q 1

A C compiler written in Rust used to be a graduate thesis. It isn't anymore.

Steve Klabnik, a 13-year Rust veteran who co-wrote The Rust Programming

Language, built a new systems language called Rue in two weeks with Claude. Roughly 70,000 lines of Rust. His own words:

I've gotten further in the two weeks I've been working on it this time than the month or two I spent last time.

Andreas Kling, creator of the Ladybird browser and a career C++ engineer, ported Ladybird's JavaScript engine from C++ to Rust in two weeks by directing Claude Code and Codex through hundreds of small prompts. Roughly 25,000 lines of Rust, byte-for-byte parity with the C++ original, zero regressions across 65,000+ combined test262 and Ladybird tests.

"The same work would have taken me multiple months to do by hand."

None of this was possible in 2024. In 2025 it was marginal. In early 2026 it's becoming commonplace

So much for "but the ecosystem"

The strongest argument for Python and JavaScript was never the languages themselves. It was the ecosystems: FastAPI, Django, PyTorch, React, Next.js, npm's 4 million packages. "Our team ships features in days because the ecosystem already solved 90% of every problem." That was decisive for the last 10 years. It's been quietly eroding for the last 2.

When you ``import pydantic``, the entire validation core is a Rust library. Polars, the pandas alternative, is Rust. Hugging Face tokenizers is Rust. orjson is Rust. The JetBrains 2025 Python survey caught the telemetry: Rust usage for Python binary extensions jumped from 27% to 33% in one year.

The Python ecosystem is increasingly a Rust ecosystem wearing a Python hat.

Q 1

The plumbing is following the same trajectory. Astral, founded by Charlie Marsh in 2022, shipped ruff, uv, and ty. All three are written in Rust, and all three went from zero to hundreds of millions of monthly downloads. On March 19, 2026, OpenAI acquired Astral; the internal justification was that uv saves Codex roughly one million minutes of compute per week. Ten weeks earlier, Anthropic acquired Bun (7M monthly downloads, 89K GitHub stars) and framed it as "essential infrastructure for AI-led software engineering." Evan You's VoidZero shipped Rolldown-Vite, a Rust bundler that cut GitLab's 2.5-minute build to 40 seconds with 100x less memory.

Lee Robinson, VP of Product at Vercel: “We’ve reached peak optimization with JS.”

What’s left of “but the ecosystem” is this: the packages you import in Python and JavaScript are increasingly wrappers around code written in languages you were told you couldn’t ship in. Now you can ship in those languages directly, and the wrapper is starting to look like overhead.

Why patch when you can port?

The old open-source bargain had a positive feedback loop. You pick Python because it’s easy. You find a bug in a dependency. You fix it. You upstream the fix. The ecosystem gets healthier.

Agents broke that loop in a specific way: **the unit of contribution shifted from the patch to the port.**

Armin Ronacher, the creator of Flask, ported his Rust library MiniJinja to Go in January using an agent. The run lasted 10 hours, 3 supervised and 7 unattended. His actual human time was 45 minutes. The API cost was \$60. If porting a library across languages is a 45-minute job, the case for upstreaming a fix to someone else’s library gets weaker every month. Why patch what you can fork?

His own observation:

For me, the value is shifting from the code to the tests and documentation. A good test suite might actually be worth more than the code.

Top highlight

The loop that built PyPI and npm still works today. It is not obvious it works in 2028.

Where this argument breaks

This isn’t a clean sweep. A few things are worth admitting.

First, sometimes the right answer is still the old answer. Prisma removed its Rust query engine in favor of a TypeScript/WASM core: bundle size dropped 85%, queries got up to 3.4x faster. Native Rust binaries are hostile to serverless runtimes. PyTorch still owns roughly 85% of deep-learning research, and that isn’t changing because the model weights don’t care what language you wrap around them.

Secondly, AI is not equally good at every systems language. Smaller languages like Zig, Haskell and Gleam don't have the same quality when AI-generated (for now).

Training data shapes what the model can help you with. Rust and Go won the lottery because they were popular enough to flood GitHub. Zig, Haskell and Gleam are still on the wrong side of that curve.

The reason the shift is permanent

The old defense of Python and TypeScript was really a defense of the developer experience. Those languages were chosen because they minimized the friction between a human's idea and a shipped product. Rust was never slower at runtime; it was slower at 2am when you had to ship.

Agents do the hard part now.

The human's job shifted from "writing the code" to "architecting the system and reviewing the output." In that workflow, Python's ergonomic advantages matter less every quarter, and a harder language's runtime advantages compound every day you run the service in production.

Q 1

Armin Ronacher, in his February essay [A Language For Agents](#):

The biggest reason new languages might work is that the cost of coding is going down dramatically. The result is the breadth of an ecosystem matters less.

The last twenty years of language choice were shaped by a single constraint: humans write the code, and humans are slow at low-level languages. That constraint is gone. Stack Overflow's 2025 survey had Rust as the most-admired language for the tenth consecutive year at 72%, with Gleam at 70%, Elixir at 66%, and Zig at 64%. The stated preference was always there; the tooling finally caught up to the preference.

Karpathy framed the broader picture [in February](#):

"LLMs change the whole constraints landscape of software completely. Hints of this can already be seen, e.g. in the rising momentum behind porting C to Rust." He added that "even Rust is nowhere near optimal for LLMs as a target language."

Today's winner is the opening move, and the endgame is somewhere further out.

The cleanest statement of the new regime came from @RealRichomie on X on April 24:

The future of programming won't be languages easiest for humans. It'll be languages easiest for agents. We just shipped a Mac app where our engineers didn't know a single line of Rust (or Tauri) beforehand. Result: ~1/10th the size... highly performant. Agents are the new programmers.

A shipped app, in a language nobody on the team knew, one-tenth the size of the Electron version, faster at runtime. The humans never had to learn Rust to get there.

The next project you start doesn't have to default to Python.

Programming

Rust

Technology

Software Engineering

Artificial Intelligence



Written by Noah Mitchem

421 followers · 5 following

Follow

Building AI coding tools. Ex-IBM. Writing about the agent-first development stack.

